WaterManagement — implementation roadmap

We will start with one station and add the remaining parts once its basic behaviour works. Central/FO and Engine/Monitor can be developed in parallel after agreeing on the messages between them. Each phase includes checks before connecting it to the next part.

1. Set up the project and agree on the interfaces

- Review the assignment and divide the work: Central and FO in Python, Engine and Monitor in C++.
- Set up the repository, dependencies and a basic executable for each module. Add Dockerfiles early so we can check the build environment as we go.
- Agree on NAP and HCP: connection roles, station identity, message boundaries, replies and error cases. Write down the formats before joining the modules.
- Define the Kafka topics and JSON fields, including IDs, operation codes, recipients and measurement units.
- Use CMake and ninttp for the station modules, and check native builds on Windows and Linux before too much code depends on the platform.

2. Get one watering station working locally

- Implement the simulated valve, flow meter and leak detector. Allow manual start, stop, fault and repair from Engine's terminal.
- Track elapsed time and delivered litres, including automatic stopping when an irrigation has a duration limit.
- Connect Monitor to Engine over HCP. Keep their station ID consistent and check health periodically without blocking the local controls.
- Add timeouts and reconnection. Check that losing Monitor stops watering and that repairing a leak does not restart it by itself.
- Test the simulation and socket exchanges with local peers before connecting the station to Central.

3. Add Central registration and station supervision

- Implement Central's NAP listener and the registration and activation steps. Validate incoming frames and return a reason when a request is rejected.
- Store stations and operators in SQLite. Handle several connections without allowing worker threads to interfere with database writes.
- Have Monitor report Engine health to Central while continuing its local checks independently.
- Maintain the station states in Central, including leaks and disconnections. Check registration, repeated connections and recovery after a temporary loss of contact.

Milestone 1 — One registered and supervised station

- Central registers and activates a station, then receives its health reports.
- A simulated leak is visible in Engine, Monitor and Central, and repair is reported without restarting the whole system.

4. Connect irrigation requests through Kafka

- Implement FO's station list and request menu, then add request-file playback.
- Let Central check requests and forward accepted commands to the right station. Keep start, stop, block and activation as separate actions.
- Have Engine confirm commands, publish current flow and send the final litres and duration when irrigation ends.

- Match replies to requests and deliver notifications to the original operator. Check routing with several station and operator IDs.
- Try manual, FO and Central requests. Include rejected requests and commands received while the station is already busy or has a fault.

Milestone 2 — A complete irrigation request

- FO can request a timed irrigation and receive its confirmation, live readings and final summary through Central.
- A second operator or station can run alongside it without receiving the wrong commands or results.

5. Handle failures and finish the interfaces

- Save station measurements and pending results so an Engine restart does not lose the irrigation summary or reopen the valve.
- Retry messages after a Kafka interruption and handle repeated commands without repeating the valve action.
- Test leaks and separate failures of Monitor, Engine, FO and Central. Check which parts should continue and which need to stop or wait for reconnection.
- Keep a deliberate block in place through repair and restart. Check that late health reports or results do not overwrite the current station state.
- Add Central's dashboard and Monitor's optional Qt panel. Keep status readings current, make errors readable and put detailed diagnostics in log files.
- Repeat the station checks on Windows and Linux, including the terminal and graphical versions of Monitor.

6. Prepare deployment and the demonstration

- Deploy Central and Kafka in the cloud, with persistent storage for SQLite. Check the public endpoints from the machines used for the demonstration.
- Prepare separate Engine and Monitor containers, with persistent station data and logs. Use a different identity and state volume for each station.
- Rehearse the required setup with two operators and two stations. Run normal irrigation, stop, block/activate, leak/repair and component failure scenarios.
- Build the images and binaries before the assessment. Write down the startup commands and prepare the report, screenshots and prompt records.

Optional work — pending

- ☐ Export irrigation summaries to CSV so they can be reviewed outside the terminal.
- ☐ Add a small flow-history graph to the operator interface.

Leave these until the required requests, failure handling and deployment work.